- *Private environment variables*—These are accessed only on the server, prefixed with `NEXT_PRIVATE_` or without the `NEXT_PUBLIC_` prefix. These should store sensitive information like API keys, database credentials, and other secrets.
- *Public environment variables*—Prefixed with `NEXT_PUBLIC_`, these are exposed to the browser and embedded in the client JavaScript bundle. This means that they will be visible to anyone inspecting your application's source code.

The best practice for managing environment variables is to inject them at run time using a deployment provider like Vercel. Thus, we do not hardcode any variables in the repo or any file, even if they have default values. Instead, we allow them to be configured in the Vercel dashboard, ensuring that sensitive information is not hardcoded in your source files. The steps involved are discussed later in this chapter when we talk about deployments.

Now, to clarify the need for API key management and how to control keys. There are two strategies here for managing the API keys: application-level API keys and user-provided API keys.

9.4.2 Application-level API keys

Store application-level API keys, such as those used for OpenAI, in private environment variables to prevent hardcoding them into your source code. Additionally, make authenticated requests through server routes instead of exposing keys to the client, which minimizes the risk of unauthorized access.

Next.js facilitates this process with the `use server` directive, which designates a function or file to be executed on the server. This allows developers to securely handle sensitive operations without exposing API keys or other critical information to the client.

For instance, consider the example with the Redis client. The following code runs exclusively on the server, and sensitive information like the API tokens is never exposed to the client:

```
'use server';
import { Redis } from '@upstash/redis';

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL,
  token: process.env.UPSTASH_REDIS_REST_TOKEN,
});
```

Here, the Redis client operates on the server, and any interactions with the Redis database are conducted securely without ever revealing connection details to users.

9.4.3 User-provided API keys

For user-provided API keys, such as those from users accessing their OpenAI accounts, there are two primary approaches: server storage and client storage. When using server storage, encrypt user-provided API keys and store them in a secure database. This